IEEE INFOCOM 2024 - IEEE Conference on Computer Communications | 979-8-3503-8350-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/INFOCOM52122.2024.10621305

EdgeTimer: Adaptive Multi-Timescale Scheduling in Mobile Edge Computing with Deep Reinforcement Learning

Yijun Hao†, Shusen Yang*, Fang Li†, Yifan Zhang†, Shibo Wang†, and Xuebin Ren†
†*Xi'an Jiaotong University, China
Email: yijunhao@stu.xjtu.edu.cn, shusenyang@mail.xjtu.edu.cn, lifang1999@stu.xjtu.edu.cn,
john_efan@stu.xjtu.edu.cn, wshb20081996@stu.xjtu.edu.cn, xuebinren@mail.xjtu.edu.cn
*Corresponding Author

***Abstract*—In mobile edge computing (MEC), resource scheduling is crucial to task requests' performance and service providers' cost, involving multi-layer heterogeneous scheduling decisions. Existing schedulers typically adopt static timescales to regularly update scheduling decisions of each layer, without adaptive adjustment of timescales for different layers, resulting in potentially poor performance in practice.**

We notice that the adaptive timescales would significantly improve the trade-off between the operation cost and delay performance. Based on this insight, we propose EdgeTimer, the first work to automatically generate adaptive timescales to update multi-layer scheduling decisions using deep reinforcement learning (DRL). First, EdgeTimer uses a three-layer hierarchical DRL framework to decouple the multi-layer decision-making task into a hierarchy of independent sub-tasks for improving learning efficiency. Second, to cope with each sub-task, EdgeTimer adopts a safe multi-agent DRL algorithm for decentralized scheduling while ensuring system reliability. We apply EdgeTimer to a wide range of Kubernetes scheduling rules, and evaluate it using production traces with different workload patterns. Extensive trace-driven experiments demonstrate that EdgeTimer can learn adaptive timescales, irrespective of workload patterns and built-in scheduling rules. It obtains up to $9.1\times$ more profit than existing approaches without sacrificing the delay performance.

***Index Terms*—Mobile edge computing, resource scheduling, adaptive timescales, reinforcement learning**

I. Introduction

Efficiently utilizing edge resources—which are often more limited compared to cloud resources—matters for optimizing service performance, operation costs, and reliability in mobile edge computing (MEC) [1], [2]. Resource scheduling is a critical technique to achieve this. It works by providing a set of scheduling decisions to assign available resources to task requests *properly* [3]. In the context of MEC, the resources of clouds and edges should be viewed as a unified whole that can be jointly scheduled, while they are commonly diversified in terms of capacity and duty [4]. Efficient scheduling decisions are expected to exploit the collaboration of cloud servers, neighbor under-utilized edge servers, and the local edge server. Typically, a three-layer heterogeneous architecture is presented by segregating the overall scheduling process into layers of scheduling decisions performed in different collaboration manners, including *Layer-1: edge-cloud scheduling, Layer-2: edge-edge scheduling, and Layer-3: intra-edge scheduling* [5].

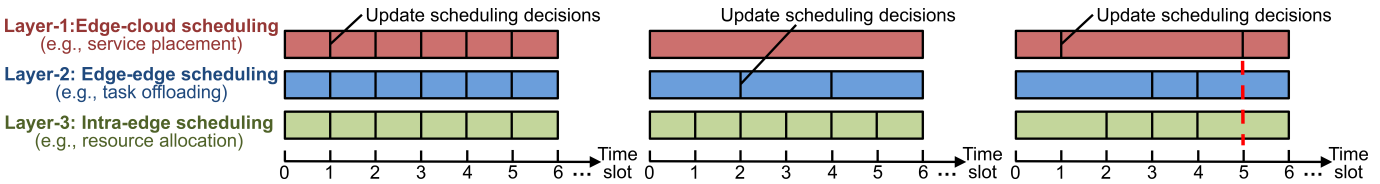
The three layers of scheduling decisions *are not executed in isolation*. When processing a task request, there is a specific sequence and correlation involved in the decision-making process for different layers. For example, edge-cloud service placement [6], edge-edge task offloading [7], and intra-edge resource allocation [8] successively determine which cloud services to pre-place in which edge servers before task arrivals, which servers to offload arrived tasks, and how many resources to allocate to offloaded tasks, respectively.

Most MEC schedulers [4], [9]–[11] update scheduling decisions at a *fixed* interval that is *uniform* across all three layers, which we call *static single-timescale mode* (see Fig. 1(a)). Typically, the updating interval is extremely short (e.g., one second) to ensure a timely decision adjustment based on real-time requests. However, such frequent decision adjustment incurs high operation costs for service providers, such as costs of service migration [12], handover [13], and resource reconfiguration [14].

Fewer schedulers [6], [15] set up different updating intervals across multiple scheduling layers, but the intervals are still *fixed*, which we call *static multi-timescale mode* (see Fig. 1(b)). They typically use a *constant* scaling ratio to determine the difference in timescales between adjacent scheduling layers, and consequently cannot adapt to environment variations. For example, in Fig. 1(b), updates in Layer-3 occur $2\times$ more frequently than in Layer-2 and $6\times$ more frequently than in Layer-1. Furthermore, human-based tuning is heavily required to identify an appropriate scaling ratio that enables service providers to save costs while providing high-performance services. It is a time-consuming and uneconomical process due to the additional labor costs involved.

That naturally raises a new critical question: *How to automatically and dynamically determine respective timescales of multiple layers of scheduling decisions to achieve the trade-off between the operation cost and service performance?* To solve this problem, we propose EdgeTimer, an *adaptive multi-timescale* scheduling framework for edge-cloud, edge-edge, and intra-edge scheduling, as shown in Fig. 1(c). EdgeTimer

Authorized licensed use limited to: China University of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.



(a) Static single-timescale scheduling. (b) Static multi-timescale scheduling. (c) Adaptive multi-timescale scheduling.

Fig. 1. Timescales of different resource scheduling frameworks in MEC. The red dotted line shows the asynchrony nature of the adaptive multi-timescale framework, i.e., EdgeTimer, where the higher-layer scheduling decision is updated but lower-layer decisions remain unchanged.

has the following three desired features:

F1: Adaptation to environment variations. The timescale for each layer of scheduling decisions is time-varying, adapting to changeable patterns of online task requests.

F2: Asynchrony over multi-layer scheduling. It decouples multi-layer scheduling decisions, allowing lower-layer decisions to remain unchanged while higher-layer decisions are updated, as shown by the dotted red line in Fig. 1(c). The value of timescales is determined independently across different scheduling layers, instead of being strongly linked by a scaling ratio.

F3: Autonomy for edge servers. To reduce communication cost from the cloud to edges, e.g., high transmission delay, each edge server is required to determine its timescales *locally*, without resorting to the computation of remote clouds.

EdgeTimer uses deep reinforcement learning (DRL) [16] to learn update policies, i.e., deciding whether to update current scheduling decisions of each layer for each edge server at each slot. DRL is well-suited to our problem as it allows to learn update policies purely through the actual network conditions and task requests. However, when applying off-the-shelf DRL to our problem, two main challenges need to be addressed.

First, conventional DRL algorithms are designed to achieve a high-level aggregated goal. In our problem, we define the overall goal as *service providers' profit* that explicitly measure the cost-performance trade-off. Correspondingly, the learning task is set to simultaneously learn three-layer policies to jointly maximize the expected profit. This makes the learning task complicated and intractable, as improper policies of any layers may lead to poor profit performance due to inter-layer dependencies. We design a *three-layer hierarchical DRL framework* to decompose the overall learning task into three independent hierarchical sub-tasks based on temporal abstraction, each of which is associated with an individual sub-goal and is handled by a DRL controller. This approach significantly reduces the task complexity, and improves learning efficiency by mitigating the effect of any layers of improper policies.

Second, conventional DRL algorithms usually use one agent for centralized decision-making, and are incapable of decentralized scheduling tasks due to non-stationary environment issues [17]. To this end, for each layer of DRL controllers, we design a *safe multi-agent DRL approach* to learn layer-specific decentralized policies. To tackle the lack of global information, the non-local information from other agents is allowed to use for ease of offline training, while it is not observed in the online process. Moreover, to ensure the reliability of online decisions, EdgeTimer also integrates a safe learning scheme to mask unsafe decisions, e.g., inter-procedure conflicting decisions, caused by uncertainty of neural networks.

We integrated EdgeTimer with a realistic Vienna system-level simulator [18] and evaluated it using multi-pattern workload traces derived from Alibaba's production clusters [19]. A total of 45 typical scheduling rules were implemented to act as built-in scheduling rules, including Kubernetes rules [20] and other widely-used rules. Extensive trace-driven experiments show that by applying EdgeTimer on 45 typical scheduling rules, it improves the service providers' profit by up to $9.1\times$ over existing schedulers under the same rule, without sacrificing the delay performance of task requests.

To our knowledge, EdgeTimer is *the first to automatically and collaboratively control the update timescales of multiple scheduling decisions to optimize the profit.* Service providers would benefit from our work in three ways: 1) They can directly improve their profit by providing high-performance services with no unnecessary costs. 2) EdgeTimer is not intrusive to current scheduling approaches of service providers. It only provides an update signal when it is necessary. Service providers can decide whether to rely on the signal based on their extensive experience. 3) Besides the application case in this paper, they can extend EdgeTimer to other single or multiple layers of scheduling decisions by simply changing the state and reward settings of DRL controllers.

II. Motivating Example

EdgeTimer introduces a new dimension (i.e., updating timescales) to the research issue of MEC scheduling, in order to further improve the trade-off between system performance and costs. We used the *profit* (detailed in Sec. III-C) as a metric to judge the advancement of performance-cost trade-offs. Figure 2 provides an example to demonstrate that by adding the EdgeTimer's features, i.e., adaptation (**F1**), asynchrony (**F2**), and autonomy (**F3**), to the existing approaches step by step, the profit can be improved significantly.

We consider two edge servers and a cloud server in a MEC system. Each of the edge servers performs three layers of scheduling decisions, including Layer-1: edge-cloud service placement, Layer-2: edge-edge task offloading, and Layer-3: intra-edge resource allocation. Figure 2(a) visualizes the updating timescales and scheduling results of state-of-the-art schedulers, where a fixed scaling ratio is used to determine the difference in updating timescales between adjacent scheduling layers. For a fair comparison, we iterate through all possible

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

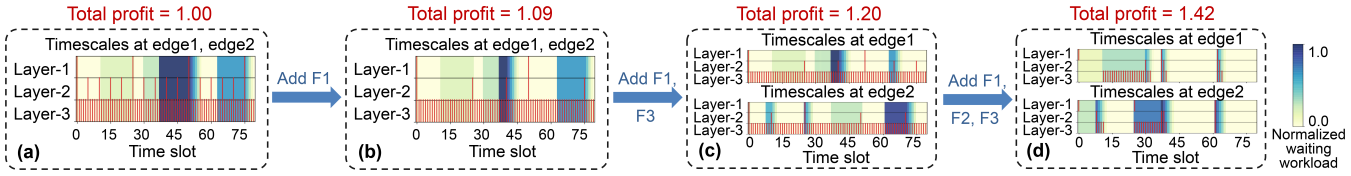


Fig. 2. An example to show that by *cumulatively* applying EdgeTimer's features of adaptation (F1), asynchrony (F2) and autonomy (F3) to existing approaches, the profit can be improved significantly. The scheduling decisions are updated at the moment of the red vertical lines.

scaling ratios in a given set to select the optimal one as the scaling ratio used in Fig. 2(a). All edge servers adopt the same timescales. The total profit obtained by the state-of-the-art scheduler is normalized to one.

In Fig. 2(b), we adjust the inter-layer scaling ratio based on the total waiting workloads of two edge servers, i.e, applying the adaptation feature of EdgeTimer (**F1**). Fig. 2(b) shows that by employing a lower decision update frequency (the sparser red vertical lines), a lower waiting workload than Fig. 2(a) can be attained (the shorter dark purple blocks). The adaptive updating timescales significantly reduce unnecessary operation costs, achieving $1.09\times$ higher profit than state-of-the-art schedulers. On this basis, the profit can be improved if we follow the autonomy feature of EdgeTimer (**F3**). Each edge server is allowed to use differentiated timescales according to local waiting workloads (Fig. 2(c)). The total profit is $1.2\times$ higher than state-of-the-art schedulers.

Notably all the above practices rely on inter-layer scaling ratios to control three-layer scheduling simultaneously. Suggested by the asynchrony feature of EdgeTimer (**F2**), the profit can be improved by updating three-layer scheduling decisions asynchronously. We set the workload threshold for each layer and update the corresponding decisions only when the current workload exceeds the layer-specific threshold (Fig. 2(d)). Finally, the approach that integrate all EdgeTimer's features makes $1.42\times$ higher profit than state-of-the-art schedulers.

Though this example demonstrates the potential of using EdgeTimer to improve the profit, its effectiveness highly relies on the prior knowledge. Once the workload pattern changes, we have to tune parameters, e.g., workload threshold, for high performance. In this paper, driven by DRL, EdgeTimer is able to automatically determine timescales, and as a result, it achieves much higher profits than all the above practices.

III. System Model and Problem Formulation

Our objective is to optimize the overall profit by updating scheduling decisions with an *adaptive* timescale in heterogeneous MEC systems.

A. MEC System Model

As shown in Fig. 3, we consider a MEC system proceeding in discrete time slots $\mathcal{T} = \{0, 1, ..., t, ...\}$. N edge servers, $\mathcal{N} = \{1, ..., N\}$, are located at respective base stations, each of which is equipped with limited computation, storage, and communication resources. All edge servers are connected via back-haul links [6]. A cloud o with sufficient resources is connected to edge servers by core networks [4].

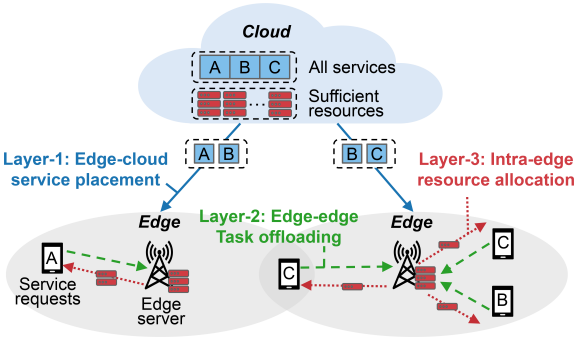


Fig. 3. The illustration of three-layer heterogeneous scheduling in MEC.

B. Resource Scheduling Model

At each time slot t, task requests arrive at edge servers stochastically. Efficient processing of tasks relies on three-layer heterogeneous scheduling with different collaborations, including *Layer-1: edge-cloud scheduling, Layer-2: edge-edge scheduling, and Layer-3: intra-edge scheduling* [5]. We apply EdgeTimer to three typical cases corresponding to the above three layers, i.e., edge-cloud service placement [6], edge-edge task offloading [7], and intra-edge resource allocation [8].

1) Layer-1: Edge-Cloud Service Placement: Each task request is associated with an application, e.g., machine learning inference or augmented reality. To process a task, the edge server must pre-cache the application-specific service, e.g., trained models and related databases [11]. Different services occupy different storage spaces. The cloud caches all services $\mathcal{S} = \{1, ..., S\}$. Considering storage limits of edge servers, only a subset of services can be placed on each of them at a given slot. Service placement provides decisions $x_{i,s}(t)$ indicating whether service s is placed on edge i at slot t.

2) Layer-2: Edge-Edge Task Offloading: To better utilize limited edge resources, tasks arrived at the local edge server i can be offloaded to an under-utilized neighbor edge j that has cached corresponding services. Decisions of task offloading $y_{i,j,s}(t)$ determine whether a task of service s is offloaded from edge server i to server j at slot t. The offloading process consumes the communication resources between the original and destination servers, and incurs transmission delay.

3) Layer-3: Intra-Edge Resource Allocation: It focuses on how much computation resources are allocated by edge i to offloaded service tasks s at slot t, denoted by $z_{i,s}(t)$. There may exist waiting queues in the buffer of edge servers. Thus, the computation delay required for allocation decisions includes the queueing delay and execution delay.

Please note that the three-layer scheduling decisions, i.e.,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

$x_{i,s}(t)$, $y_{i,j,s}(t)$ and $z_{i,s}(t)$, are *tightly coupled*. The higher-layer decisions determine lower-layer feasible regions, e.g., the tasks can only be offloaded to servers that have placed corresponding services. In contrast, the lower-layer scheduling results reflect the effectiveness of higher-layer decisions.

C. Profit Model

The profit earned by service providers can be defined by subtracting their operation cost from their revenue.

1) Revenue: Service providers get revenue by selling computation resources to users [22], [23]. Considering that delayed tasks have a negative impact on the revenue, we use a more realistic metric to represent the revenue obtained by server i at slot t as:

$$g_i(t) = \sum_{s \in \mathcal{S}} \mu_{i,s}(t) p_s(t), \tag{1}$$

where $\mu_{i,s}(t)$ is the total amount of computation resources allocated to service tasks s that were *processed within delay budgets* by server i at slot t. Given that servers are allowed to differentiate the unit price of each service task for profit maximization [24]. Furthermore, the unit price usually decreases as the billing cycle increases in real markets [25]. We denote the unit price of computation resources using a service-specific time-varying parameter $p_s(t)$.

2) Operation Cost: Too frequent adjustment of scheduling decisions, i.e., $x_{i,s}(t)$, $y_{i,j,s}(t)$, and $z_{i,s}(t)$, would increase operation cost. Specifically, placing a service s requires the edge server i to download non-trivial data from server j of a neighbor edge or a remote cloud, which incurs the operation cost, $C^1_{i,j,s}$, defined by the service type and transmission distance [6]. Following a general assumption [6], removing an installed service does not increase the cost. The cost of placing a new service s on edge i is $C^1_{i,s} = \min_{j \in \{j | x_{j,s}(t-1)=1\}} C^1_{i,j,s}$. The operation cost for service placement of edge i at slot t can be expressed as:

$$c^1_i(t) = \sum_{s \in \mathcal{S}} C^1_{i,s} \mathbb{I}_{\{1\}}(x_{i,s}(t) - x_{i,s}(t-1)), \tag{2}$$

where $\mathbb{I}_{\{1\}}(x)$ is an indicator function that has value of 1 if $x \in \{1\}$ and 0 otherwise. Furthermore, if a task is offloaded from a current server k to another server m, it incurs operation cost $C^2_{k,m,s}$ including data transmission cost and service interruption cost caused by the inter-server handover [13]. The offloading cost of edge i at slot t is defined as:

$$c^2_i(t) = \sum_{s \in \mathcal{S}} C^2_{k,m,s} \mathbb{I}_{\{1\}}(y_{i,k,s}(t-1) y_{i,m,s}(t)). \tag{3}$$

Changing resource allocation decisions may lead to resource reconfiguration, and incurs the resource lead time caused by the server booting and initialization [14], [26]. Since shutting down a server is much faster than booting it, we use the increase in resources allocated as the operation cost of resource reconfiguration. Let C^3_i be the one-unit reconfiguration cost of the computation resource. At time slot t, the allocation cost of edge i can be represented as:

$$c^3_i(t) = \sum_{s \in \mathcal{S}} C^3_i [z_{i,s}(t) - z_{i,s}(t-1)]^+. \tag{4}$$

D. Problem Formulation

Our objective is to *maximize the long-term profit* by jointly optimizing timescales that are used to update three-layer scheduling decisions. The control variables are $a^1_i(t)$, $a^2_i(t)$, and $a^3_i(t)$, indicating *whether current placement, offloading, and allocation decisions at edge server i need to be updated, respectively*. Existing scheduling algorithms act as built-in rules to provide candidate scheduling decisions for placement $\widetilde{x}_{i,s}(t)$, offloading $\widetilde{y}_{i,j,s}(t)$, and allocation $\widetilde{z}_{i,s}(t)$. For example, if $a^1_i(t) = 1$, the server i would perform placement decisions $\widetilde{x}_{i,s}(t)$ generated by existing placement rules. Conversely, the server i would continue previous decisions. The placement decisions performed in edge i at slot t can be expressed as:

$$x_{i,s}(t) = \Gamma^1(a^1_i(t)) = \begin{cases} \widetilde{x}_{i,s}(t), & a^1_i(t) = 1, \\ \Gamma^1(a^1_i(t-1)), & a^1_i(t) = 0. \end{cases} \tag{5}$$

The joint optimization problem can be formulated as:

$$\begin{aligned} \widetilde{\mathcal{P}}: \max\ & \mathbb{E}[\sum_{t=0}^{\infty} \sum_{i \in \mathcal{N}} (g_i(t) - a^1_i(t) c^1_i(t) - a^2_i(t) c^2_i(t) - a^3_i(t) c^3_i(t))] \\ \text{s.t.}\ & a^1_i(t), a^2_i(t), a^3_i(t) \in \{0,1\}, \forall t, i. \end{aligned} \tag{6}$$

IV. EdgeTimer Design

In EdgeTimer, we design a three-layer HDRL framework to decouple the joint optimization problem. Then, the safe MADRL algorithm for each sub-problem is explained.

A. Three-Layer Hierarchical DRL for Problem Decomposition

Achieving the objective in (6) relies on performing three layers of tightly coupled decision-making. This makes it intractable to generate three effective decisions simultaneously. For ease of learning, we decouple the original problem into three independent sub-problems, and design a three-layer hierarchical deep reinforcement learning (HDRL) framework to solve different sub-problems in a coordinated manner.

1) Hierarchical Decomposition for the Original Problem: We decouple the original problem ($\widetilde{\mathcal{P}}$) into sub-problems ($\mathcal{P}1, \mathcal{P}2, \mathcal{P}3$) by decomposing the total objective into respective sub-revenue and sub-cost of different layers of scheduling.

Edge-cloud service placement sub-problem ($\mathcal{P}1$). The placement cost is derived from (2). Although the revenue cannot be directly decomposed, the extent of service coverages can potentially measure the contribution of current placement decisions to the total revenue, as verified in subsection VI-D. The service placement sub-problem can be formulated as:

$$\begin{aligned} \mathcal{P}1: \max\ & \mathbb{E}[\sum_{t=0}^{\infty} \alpha d(t) - \sum_{t=0}^{\infty} \sum_{i \in \mathcal{N}} a^1_i(t) c^1_i(t)] \\ \text{s.t.}\ & a^1_i(t) \in \{0,1\}, \forall t, i, \end{aligned} \tag{7}$$

where $d(t)$ is the amount ofcomputation demands of tasks covered by placed services at slot t. It is associated with current placement decisions, i.e., $d(t) = d(\Gamma^1(a^1_1(t)), ..., \Gamma^1(a^1_N(t)))$. The coefficient α is to weigh the impact of service coverages.

Edge-edge task offloading sub-problem ($\mathcal{P}2$). Given the offloading sub-cost in (3), a straight option to formulate

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

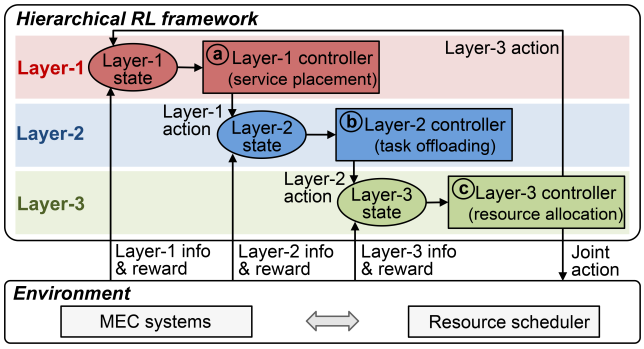


Fig. 4. The workflow of three-layer hierarchical DRL framework.

the offloading sub-problem is to set the transmission delay budget and maximize the difference between total computation demands of current tasks, $u(t)$, that complete offloading within the transmission delay budget and the sub-cost. However, the above option is not equivalent to maximizing the original profit, as it may result in offloading too many workloads to the nearest edges, inducing high computation delay and revenue loss. To avoid load imbalance, we introduce a load balancing function $v(\cdot)$ that maps current offloading decisions $\Gamma^2(a_i^2(t))_{i\in\mathcal{N}}$ to the workload variance of all edge servers. The task offloading sub-problem can be given by:

$$\mathcal{P}2: \max \mathbb{E}[\sum_{t=0}^{\infty} \beta u(t) - \sum_{t=0}^{\infty} \sigma v(t) - \sum_{t=0}^{\infty} \sum_{i\in\mathcal{N}} a_i^2(t) c_i^2(t)]$$
$$\text{s.t.} \quad a_i^2(t) \in \{0,1\}, \forall t, i, \tag{8}$$

where β and σ are coefficients to control the degree of the offloaded task satisfaction and workload variance, respectively.

Intra-edge resource allocation sub-problem ($\mathcal{P}3$). Similarly, we set computation delay budgets for service requests, and denote the total computation demands of current tasks processed within computation delay budgets and the corresponding weight coefficient as $l(t)$ and ϕ, respectively. The resource allocation sub-problem is defined by:

$$\mathcal{P}3: \max \mathbb{E}[\sum_{t=0}^{\infty} \phi l(t) - \sum_{t=0}^{\infty} \sum_{i\in\mathcal{N}} a_i^3(t) c_i^3(t)]$$
$$\text{s.t.} \quad a_i^3(t) \in \{0,1\}, \forall t, i. \tag{9}$$

2) Workflow of Three-Layer Hierarchical DRL Framework: Hierarchical deep reinforcement learning (HDRL) [27] is a promising framework to extend traditional DRL algorithms to learn complicated tasks, by breaking the overall decision-making down into a hierarchy of sub-tasks. As shown in Fig. 4, we design a tailored HDRL framework that comprises three-layer DRL controllers, i.e., ⓐ Layer-1 controller, ⓑ Layer-2 controller, and ⓒ Layer-3 controller, to cope with three layers of scheduling sub-problems $\mathcal{P}1$, $\mathcal{P}2$, and $\mathcal{P}3$, respectively.

At slot t, the input of each layer of the controllers is the layer-specific state composed of two components, i.e., the status of queueing tasks after executing the action of the layer above it and the current resource configuration information from the environment. The DRL controller uses the state to select the action that determines whether to update the current scheduling decisions of the corresponding layer.

After performing the joint three-layer action, the environment transitions to slot $t+1$ with new three-layer scheduling information, and provides three-layer controllers with respective rewards. The reward for each controller is set based on the targeted sub-problem's objective. The controllers use these rewards to improve their actions for future time slots. Details on states, actions, and rewards are provided in subsection IV-B.

B. Safe Multi-Agent DRL for Decentralized Scheduling

Each layer of the controllers uses the tailored multi-agent DRL algorithm to generate decentralized actions, and a safe learning scheme is applied to ensure system reliability.

1) Decentralized POMDP Modeling: To save the communication cost, each of the controllers is required to perform actions in a decentralized manner, where *each edge server makes its own decisions independently without relying on the global system information from other servers.* We formulate the learning process as a decentralized partially observable Markov decision process (Dec-POMDP) [28]. For each of the controllers, *each edge server acts as an agent* to learn decentralized actions. In addition to conventional components of a MDP, such as a state space $\mathcal{S}$, an action space $\mathcal{A}$, a state transition function $p: \mathcal{S}\times\mathcal{A}\to\mathcal{S}$, a shared reward function $r: \mathcal{S}\times\mathcal{A}\to\mathbb{R}$, and a discount factor γ, an extra local observation space $\mathcal{O}=\times_{i\in\mathcal{N}}\mathcal{O}_i$ is considered in Dec-POMDP to reflect local observations available to each agent from the global state $\mathcal{S}$.

Each agent i uses its policy $\pi_i(a_i(t)|o_i(t))$ to select the action $a_i(t)$ from the partial observations $o_i(t)$. The goal of the Dec-POMDP is to find a joint policy of all agents $\Pi = \{\pi_i(a_i(t)|o_i(t))|i\in\mathcal{N}\}$ that maximizes the expected cumulative reward:

$$\max_{\Pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(\mathcal{S}(t), \Pi(\mathcal{A}(t)|\mathcal{O}(t)))], \tag{10}$$

where $\mathcal{S}(t)$, $\mathcal{A}(t)$, and $\mathcal{O}(t)$ are global states, actions, and observations of all agents at slot t, respectively. The layer-specific state, action, and reward are summarized in Table I.

2) Safe DRL Design: In our problem, DRL may generate unsafe actions that are in conflict with the executed actions. For example, the Layer-2 action may result in tasks not being offloaded due to the fact that the Layer-1 controller has just removed the required services from the destination server. The unsafe actions result in high scheduling delay for these unoffloaded tasks. To ensure safety in DRL, we design a *discriminator-aided action masking* scheme to avoid executing unsafe actions. Specifically, we build a discriminator to check whether the current action meets safety conditions, e.g., the destination server has installed the corresponding services. For unsafe actions, their associated logits, i.e, raw outputs of neural networks, are replaced by large negative numbers, thereby the probability of selecting these unsafe actions becomes virtually zero. In this way, agents will replace the original actions with "update decision" actions before applying them

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SETTINGS OF DRL COMPONENTS FOR EACH LAYER OF THE CONTROLLERS CONSIDERED.

Layers of DRL controllers	States of each agent	Actions of each agent	Rewards for all agents
Layer-1 controller edge-cloud service placement	extent of service coverages, available memory, installed service types, service types of arrived tasks	whether to update the current placement decisions of its own agent	$\sum_{i\in\mathcal{N}}[(a\cdot\text{PlaceCost}_i +b\cdot(\text{TaskUnserved}_i-\text{DudgetUnserved}))^{-1}]$
Layer-2 controller edge-edge task offloading	ratio of tasks that violate delay budgets, available CPU unit, amount of unoffloaded workloads	whether to update the current offloading decisions of its own agent	$\sum_{i\in\mathcal{N}}[(c\cdot\text{OffloadCost}_i+d\cdot(\text{TaskDelay}_i -\text{DudgetDelay}))^{-1}]+e\cdot\text{WorkloadVariance}$
Layer-3 controller intra-edge resource allocation	ratio of tasks that violate delay budgets, rate variance among tasks, available CPU unit, workloads waiting for resource allocation	whether to update the current allocation decisions of its own agent	$\sum_{i\in\mathcal{N}}[\text{UndelayedWorkload}_i/(f\cdot\text{AllocateCost}_i +g\cdot(\text{TaskDelay}_i-\text{DudgetDelay}))]$

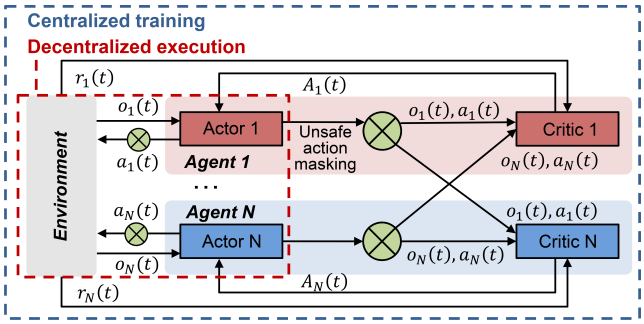


Fig. 5. The illustration of safe multi-agent DRL algorithm.

to the system. Then, EdgeTimer will use built-in scheduling rules, e.g., Kubernetes rules [20], to determine safe scheduling results within the feasible region.

3) Algorithm Training for Safe Multi-Agent DRL: We integrate safe learning with multi-agent DRL (MADRL) [29] to achieve the goal of Dec-POMDP in (10). As shown in Fig. 5, we adopt a *centralized training and decentralized execution* scheme, allowing agents to use non-local information from other agents to ease offline training, while *the extra information is not allowed to use during the online inference*. Specifically, each agent has two neural networks, where an actor network learns the policy with local observations, and a critic network approximates the centralized value function from the global environment to evaluate the current policy. The training process is shown in Algorithm 1.

Decentralized execution with actors. For each agent $i\in\mathcal{N}$, its actor i maps a local observation $o_i(t)$ to an action $a_i(t)$ with policy π_{θ_i} parameterized by θ_i. To maximize the return obtained by the policy in (10), the actor for agent i is updated following the policy gradient theorem [30]:

$$\nabla_{\theta_i}J(\pi_{\theta_i})=\mathbb{E}_{\pi_{\theta_i}}[\nabla_{\theta_i}\log\pi_{\theta_i}(a_i(t)|o_i(t))A_i(\mathcal{O}(t),\mathcal{A}(t))], \quad (11)$$

where $A_i(\mathcal{O}(t),\mathcal{A}(t))$ is the centralized advantage function that receives global observations, $\mathcal{O}(t)=(o_1(t),...,o_N(t))$, together with actions of all agents, $\mathcal{A}(t)=(a_1(t),..,a_N(t))$, and outputs a measure of policy performance for agent i. The advantage function can be computed using generalized advantage estimation (GAE) [31]:

$$A_i(\mathcal{O}(t),\mathcal{A}(t))=\sum_{l=0}^{T-t}(\gamma\lambda)^l\overline{V}_i(t+l), \quad (12)$$

Algorithm 1: EdgeTimer for adaptive multi-timescale resource scheduling

/* **Initialization** */
Initialize parameters of neural networks, actions $(a_i^1(0),a_i^2(0),a_i^3(0))$, and replay buffers (R^1,R^2,R^3);
Initialize the network configuration $\widetilde{s}$;
for epoch $=0,\cdots,M$ **do**
  for time slot $t=1,\cdots,T$ **do**
    for each edge server $i=1,\cdots,N$ **do**
      /* **Generate Layer-1 actions** */
      $o_i^1(t)\leftarrow\text{STEP}(a_i^1(t-1),a_i^3(t-1))$;
      $a_i^1(t)\leftarrow\pi(o_i^1(t)|\theta_i^1)$;
      /* **Generate Layer-2 actions** */
      $o_i^2(t)\leftarrow\text{STEP}(a_i^2(t-1),a_i^1(t))$;
      $a_i^2(t)\leftarrow\pi(o_i^2(t)|\theta_i^2)$;
      /* **Generate Layer-3 actions** */
      $o_i^3(t)\leftarrow\text{STEP}(a_i^3(t-1),a_i^2(t))$;
      $a_i^3(t)\leftarrow\pi(o_i^3(t)|\theta_i^3)$;
    /* **Compute three-layer rewards** */
    Perceive rewards $r_{i\in\mathcal{N}}^1(t)$, $r_{i\in\mathcal{N}}^2(t)$, and $r_{i\in\mathcal{N}}^3(t)$;
    Store transitions of three controllers in R^1, R^2, and R^3, respectively;
    if $\text{DONE}(o_{i\in\mathcal{N}}^1(t),o_{i\in\mathcal{N}}^2(t),o_{i\in\mathcal{N}}^3(t))$ **then**
      $o_{i\in\mathcal{N}}^1(t),o_{i\in\mathcal{N}}^2(t),o_{i\in\mathcal{N}}^3(t)\leftarrow\text{RESET}(\widetilde{s})$;
  Sample mini-batch W^1, W^2, and W^3 from R^1, R^2, and R^3 as training samples, respectively;
  Update parameters of networks using (11) and (14);

$$\overline{V}_i(t+l)=r_i(t+l)+\gamma V_i(\mathcal{O}(t+l+1))-V_i(\mathcal{O}(t+l)), \quad (13)$$

where λ is the weight discount to control the trade-off between variance and bias, T is the trajectory length, and $V_i(\mathcal{O}(t))$ is the value function that indicates the expected return of starting from global observations $\mathcal{O}(t)$ under policy π_{θ_i}. We design an individual critic network for agent i to approximate $V_i(\mathcal{O}(t))$.

Centralized training with critics. The critic V_{ω_i} for agent i with parameter ω_i can be trained by minimizing the mean-square error as follows:

$$L(\omega_i)=(V_{\omega_i}(\mathcal{O}(t))-\sum_{k=0}^{T-t}\gamma^k r_i(t+k))^2. \quad (14)$$

V. IMPLEMENTATION

EdgeTimer was implemented on a realistic Vienna system-level simulator [18], embedding 45 combinations of typical

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

scheduling rules for three-layer scheduling.

A. MEC System Simulator

We implemented 12 edge servers and one cloud server in a realistic Vienna system-level simulator[1] [18] with MATLAB. In the simulation, edge servers were distributed over a 5 km$\times$5 km region. Each edge server was equipped with four CPU cores and eight GB memory. The cloud had eight CPU cores and 16 GB memory. The length of each time slot was one second. The unit price of computation resource was derived from the pay-as-you-go pricing model at Microsoft Azure [25]. To simulate operation costs, we measured the service migration time [12], handover time [13], and server lead time [14] in service placement, task offloading, and resource allocation, respectively, and converted them to the cost based on the hourly price at Microsoft Azure. For simplicity, we dropped the dependency on the money and service types and used relative weights to set the unit price, placement cost, offloading cost, and unit allocation cost as 25, $0.3d_1$, $0.1d_2$, and 0.5, respectively. Parameters d_1 and d_2 are the transmission distance of service migration and task offloading, respectively.

B. Integration With Representative Scheduling Rules

The simulator embedded a total of 45 combinations of representative scheduling rules for service placement, task offloading, and resource allocation. These built-in rules were executed when EdgeTimer generated the "update decision" action (discussed in subsection III-D). The service placement rules included a Kubernetes placement rule-horizontal pod autoscaling (HPA) [32], Top-K rule [6], and always migration rule (AM) [33]. For task offloading, we implemented 5 offloading rules of Kubernetes [20] (MRP, LRP, RLP, SSP, and RCRP). The allocation rules consisted of proportional fairness (PF) [34], round robin (RR) [35], and equal allocation (EA).

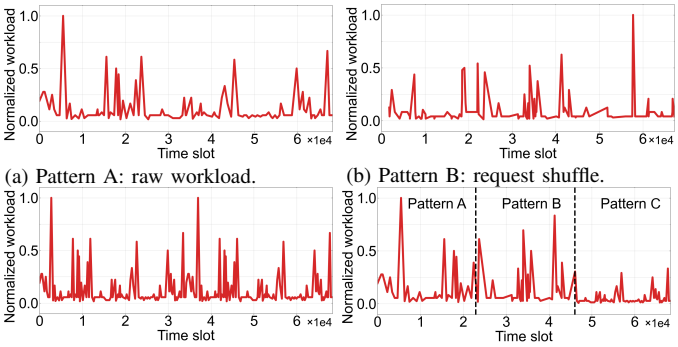
C. Scalability to Multi-Pattern Workloads

When applying DRL to task requests with varying patterns, sparse reward issues may arise, i.e., the agents rarely receive useful reward signals. For instance, when an agent selected actions during the idle period with empty queues, e.g., the night period, it would receive invalid rewards. Instead, we designed an automatic skip mechanism that allowed agents to use non-DRL-aided default actions when the request queue was empty. The default action of each edge server was set to continue its previous scheduling decisions.

D. Training Settings

The safe multi-agent DRL algorithm was implemented in Python 3.6.2 with PyTorch 1.10.2. Both the actor and critic networks included a one-layer RNN with 64 hidden units and were updated by the learning rate of 5×10^{-4}. The discount factor γ was set to 0.99, and parameter λ in GAE was 0.95.

[1]The simulator enables to perform resource scheduling of 5G multi-tier networks. We extended it to MEC system scenarios by considering each base station as an edge server and adding an extra network node as a cloud.



(a) Pattern A: raw workload. (b) Pattern B: request shuffle.
(c) Pattern C: 2$\times$ request frequency. (d) Pattern D: A-B-C combination.

Fig. 6. Illustrations of four workload patterns.

VI. Evaluation

A. Methodology

The training and inference were conducted with an Intel(R) Core CPU (2.90GHz 8-cores) and 512GB RAM.

Multi-Pattern Task Workloads. We evaluated EdgeTimer on real-world cluster traces from Alibaba [19] that provided task information per second within 8 days, including start time, end time, task type, required CPU, and required memory. We considered "task type" in traces as the type of services, and divided task requests into 12 services. The workload of each service task was estimated by multiplying the number of CPU cores it required by its actual processing time. Following the methodology in KaiS [15], the delay budget of each service task was set by proportionally scaling its actual processing time. To evaluate EdgeTimer in task arrivals with different patterns, we generated multi-pattern trace as shown in Fig. 6. The pattern A was the raw workload under stochastic arrivals (Fig. 6(a)). In pattern B, we shuffled the order in which the edge server received tasks to reflect interference from other tasks (Fig. 6(b)). Furthermore, we doubled the request frequency in pattern C to simulate an increase in workloads (Fig. 6(c)), and concatenated the above three patterns as pattern D to evaluate the adaptability to pattern changes (Fig. 6(d)).

Comparatives. We compared EdgeTimer with four comparison algorithms: *Static single-timescale (SST)* [4], [11] that updates three-layer scheduling decisions at each slot; *Static multi-timescale (SMT)* [6], [15] that updates decisions of each layer via a fixed and specific timescale[2]; *Delay-triggered update (DT)* and *workload-triggered update (WT)* that update an edge's decisions when the delay and waiting workload in the edge exceed corresponding thresholds, respectively.

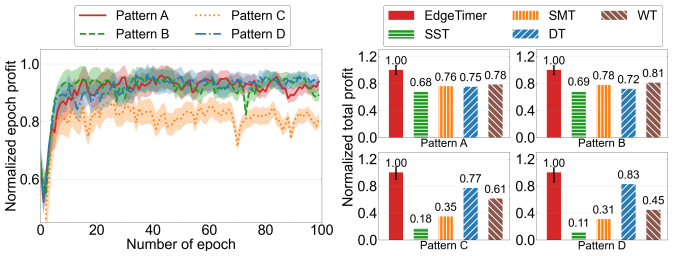
B. Performance on Multi-Pattern Workloads

We compared EdgeTimer with comparatives in four workload patterns (Fig. 6). The built-in scheduling rule used in this subsection is AM-MRP-EA, i.e., AM for service placement, MRP for task offloading, and EA for resource allocation.

Profit Performance. Fig. 7(a) shows the normalized epoch profit of EdgeTimer versus the number of offline training

[2]For each layer, we iterated through the timescale set, $\{1, 10, 50, 100\}$, to select the optimal as its fixed timescale.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 7. (a) Normalized epoch profit of EdgeTimer and (b) normalized total profit of EdgeTimer and comparatives in different workload patterns.

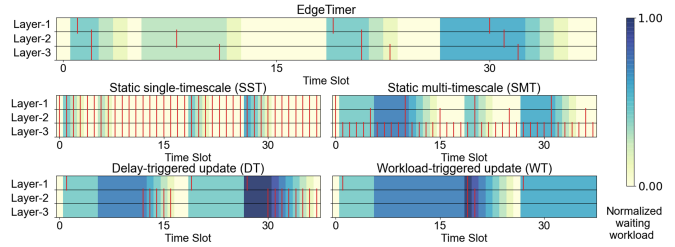


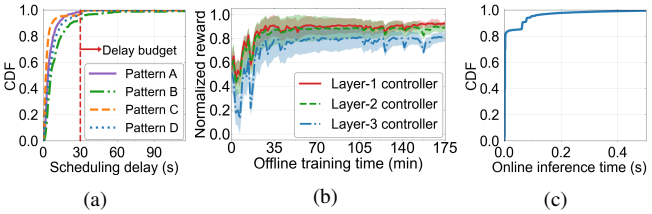
Fig. 8. Illustrations of updating timescales versus normalized waiting workloads for EdgeTimer and four comparatives. The scheduling decisions are updated at red vertical lines.

epochs in different workload patterns. The epoch profit is normalized against the profit obtained in pattern D. It shows that EdgeTimer enables to support different workload patterns and converges within 20 epochs. Even for pattern C and D where the learning task becomes more intractable due to the workload spike and the pattern fluctuating, nearly equivalent epochs are required for convergence compared with that in the raw trace. The converged profit is lowest in pattern C. This is because as the request frequency increases, more operation costs are required to complete tasks within the delay budget.

Fig. 7(b) demonstrates the online profit performance for EdgeTimer and four comparatives. The total profit is normalized against that of EdgeTimer in each pattern. EdgeTimer achieves the highest profit, irrespective of workload patterns. For pattern A, EdgeTimer stably improves the profit for $1.47\times$, $1.32\times$, $1.33\times$, and $1.28\times$ than SST, SMT, DT, and WT approaches, respectively. In particular, EdgeTimer shines in high-workload patterns (pattern C) and high-fluctuating (pattern D), where EdgeTimer achieves $5.56\times$ and $9.1\times$ higher profit than comparatives, respectively.

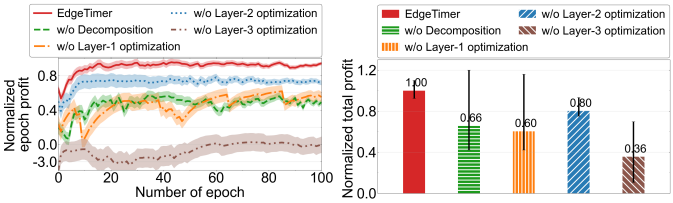
Fig. 8 investigates the underlying efforts behind EdgeTimer. It shows that the effectiveness of EdgeTimer comes from adaptively adjusting the update frequency of scheduling decisions based on the current task performance, compared with SST and SMT. Furthermore, EdgeTimer outperforms DT and WT, as it enables to mitigate potential overload in advance using more comprehensive observations, rather than relying solely on the delay or workload metrics (dense red vertical lines only appear at the end or beginning of the dark purple blocks).

Delay Performance. Given a service type, Fig. 9(a) shows the CDF of the scheduling delay for such service requests under different workload patterns. We can observe that nearly



(a) (b) (c)

Fig. 9. (a) CDF of the scheduling delay in EdgeTimer under different patterns. (b) Normalized rewards of each layer of DRL controllers in EdgeTimer. (c) CDF of the online inference time in EdgeTimer.



(a) Offline profits. (b) Online profits.

Fig. 10. Impact of the three-layer hierarchical DRL design on profit results.

all service requests can be completed within the delay budget, irrespective of workload patterns. It indicates that huge profits in EdgeTimer are not achieved at the expense of the user experience, i.e., delay performance.

Training and Inference Time. Fig. 9(b) demonstrates offline average rewards of Layer-1, Layer-2, and Layer-3 controllers in EdgeTimer. The overall controllers enable to achieve convergence within 35 minutes. Fig. 9(c) shows the CDF of the time spent by EdgeTimer to generate policies during the online process. The average inference time is 0.026 second that is less than the typical time interval between scheduling events, i.e, one second. These results indicate that EdgeTimer manages to execute scheduling policies at each time slot and imposes no extra overhead in production.

C. Performance With Different Built-In Scheduling Rules

Table II shows profit gains compared with existing approaches under 45 built-in scheduling rules in pattern A. EdgeTimer outperforms existing approaches for all built-in scheduling rules. For some Kubernetes rules, such as HPA-MRP-PF, EdgeTimer improves the profit performance for up to $2.43\times$ compared with approaches under the same rule.

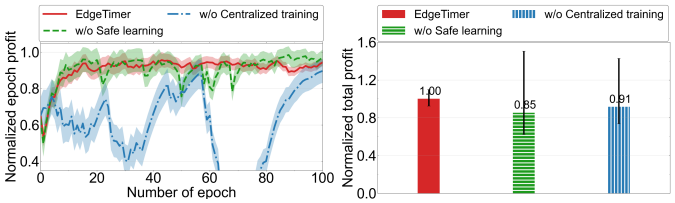
D. EdgeTimer Deep Dive

Impact of Three-Layer Hierarchical DRL Design. We design a three-layer hierarchical DRL framework to decompose the original task into three layers of sub-tasks and optimize them in a coordinated manner. To verify its contribution, Fig. 10 compared the offline and online profit of EdgeTimer, EdgeTimer without task decomposition, i.e, w/o Decomposition, and EdgeTimer without each layer of sub-task optimization in turn, i.e, w/o Layer-1 optimization, w/o Layer-2 optimization, and w/o Layer-3 optimization. All results are normalized against that of EdgeTimer. EdgeTimer accelerates the convergence in the offline process and obtains $1.52\times$ higher online profit than w/o Decomposition, since the task decomposition scheme simplifies the learning task significantly. Moreover,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PROFIT GAINS COMPARED TO EXISTING APPROACHES UNDER DIFFERENT BUILT-IN SCHEDULING RULES IN PATTERN A

		Service placement rule - resource allocation rule								
		HPA - PF	HPA - RR	HPA - EA	Top-K - PF	Top-K - RR	Top-K - EA	AM - PF	AM - RR	AM - EA
Task offloading rule	MRP	1.73-2.43x	1.70-2.39x	1.66-2.81x	1.40-3.87x	1.39-3.86x	1.40-3.55x	1.30-1.55x	1.31-1.56x	1.28-1.47x
	LRP	1.71-2.39x	1.70-2.39x	1.72-2.94x	1.34-3.85x	1.36-3.90x	1.53-3.56x	1.26-1.53x	1.25-1.52x	1.26-1.48x
	RLP	1.72-2.41x	1.71-2.40x	1.72-2.92x	1.31-3.83x	1.32-3.87x	1.48-3.60x	1.26-2.37x	1.27-2.38x	1.33-1.49x
	SSP	1.38-1.99x	1.39-2.01x	1.41-2.98x	1.91-4.67x	**1.92-4.70x**	1.29-3.61x	1.28-3.89x	1.26-3.83x	1.23-2.07x
	RCRP	1.72-2.41x	1.73-2.43x	1.70-2.92x	1.32-3.82x	1.30-3.78x	1.57-3.63x	1.26-1.59x	1.26-1.60x	1.30-1.50x



(a) Offline profits. (b) Online profits.

Fig. 11. Impact of the safe multi-agent DRL design on profit results.

removing any layer of optimization from EdgeTimer results in a decrease in profit of at least $1.25\times$, which reveals that each layer is indispensable for the overall performance.

Impact of Safe Multi-Agent DRL Design. Each task is learned by the safe multi-agent DRL algorithm, where centralized training and safe learning are two key components to learn decentralized decisions under partial observations and ensure system reliability, respectively. To verify their contributions, Fig. 11 compared the offline and online profit of EdgeTimer, EdgeTimer without centralized training, i.e, w/o Centralized training, and EdgeTimer without safe learning, i.e, w/o Safe learning. EdgeTimer manages to reach a stable high profit than w/o Safe learning as it avoids high scheduling delay caused by unsafe actions. Furthermore, EdgeTimer overcomes the instability issues during the offline process encountered by w/o Centralized training, achieving $1.1\times$ higher online profit than that of w/o Centralized training. This is because centralized training can utilize comprehensive information to evaluate policies of actor networks, providing a more precise direction for network updates.

VII. RELATED WORK

Profit Optimization in MEC. It is an important problem for service providers to maximize their profits that computed by deducting operation costs from user payments for allocated resources. Many works are developed on reducing operation costs by modifying scheduling policies for service placement [36]–[39], task offloading [12], and resource allocation [14], [26]. Another line of works aim to optimizing the price of resources based on uniform pricing [24], [40]–[42], differentiated pricing [24], [41], [42], and auction-based pricing strategies [24], [43], [44], to benefit profits. EdgeTimer opens a new dimension for profit maximization by optimizing and coordinating the update frequency of decisions in multi-layer scheduling, which enables to improve profits without changing the running scheduling algorithms of service providers.

Multi-Timescale Scheduling in MEC. Multi-timescale scheduling is often applied to scheduling problems that contain multiple sequential decision-making for stability and cost saving. Existing works adopt two timescales to update decisions for service placement and task offloading [6], [15], and task offloading and resource allocation [45]. However, update timescales of different decisions in all the above works are fixed over time, synchronous at a fixed interval, and consistent for all edge servers. EdgeTimer first provides an adaptive, asynchronous, and autonomous multi-timescale scheduling framework in edge computing.

DRL for Scheduling in MEC. DRL has been widely used in MEC to provide adaptive scheduling decisions for service placement [15], task offloading [7], [15], resource allocation [8], and server reservation [10]. EdgeTimer integrates two tailored designs with the vanilla DRL algorithm to generate adaptive timescales for multi-layer heterogeneous scheduling decisions. First, it adopts a three-layer hierarchical DRL framework to learn coupled policies of edge-cloud, edge-edge, and intra-edge scheduling for low complexity. Second, a safe multi-agent learning scheme is introduced in EdgeTimer to ensure reliability and accelerate learning.

VIII. CONCLUSION

In this paper, we study a new problem about how to update the scheduling decisions in edge computing to maximize the profit of service providers. To this end, we propose EdgeTimer, an adaptive, asynchronous, and autonomous multi-timescale scheduling framework using deep reinforcement learning. EdgeTimer enables each edge server to independently update respective decisions for edge-cloud, edge-edge, and intra-edge scheduling in a coordinated manner. We design a three-layer hierarchical DRL to learn a hierarchy of policies with low complexity, and a safe multi-agent DRL for reliability enhancement. Experiments on multi-pattern workloads demonstrate that EdgeTimer obtains up to $9.1\times$ higher profit than existing approaches and can support at least 45 scheduling rules. Service providers can use EdgeTimer as a plug-and-play approach by directly replacing scheduling rules in EdgeTimer with their own scheduling algorithms. EdgeTimer opens a new dimension for profit improvement in edge computing.

REFERENCES

[1] Y. C. Hu, M. Patel, D. Sabella, N. Sprecher, and V. Young, "Mobile edge computing—a key technology towards 5g," *ETSI white paper*, vol. 11, no. 11, pp. 1–16, 2015.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.

[3] F. Jiang, K. Wang, L. Dong, C. Pan, W. Xu, and K. Yang, "Deep-learning-based joint resource scheduling algorithms for hybrid mec networks," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6252–6265, 2019.

[4] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *IEEE INFOCOM*, 2020.

[5] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, 2021.

[6] V. Farhadi, F. Mehmeti, T. He, T. La Porta, H. Khamfroush, S. Wang, and K. S. Chan, "Service placement and request scheduling for data-intensive applications in edge clouds," in *IEEE INFOCOM*, 2019, pp. 1279–1287.

[7] R. Han, S. Wen, C. H. Liu, Y. Yuan, G. Wang, and L. Y. Chen, "Edgetuner: Fast scheduling algorithm tuning for dynamic edge-cloud workloads and resources," in *IEEE INFOCOM*, 2022.

[8] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in iot edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1133–1146, 2020.

[9] X. Wang, J. Ye, and J. C. Lui, "Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach," in *IEEE INFOCOM*, 2022.

[10] J. Zhang, S. Chen, X. Wang, and Y. Zhu, "Deepreserve: Dynamic edge server reservation for connected vehicles with deep reinforcement learning," in *IEEE INFOCOM*, 2021.

[11] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, "Joint service placement and request routing in multi-cell mobile edge computing networks," in *IEEE INFOCOM*, 2019, pp. 10–18.

[12] F. Guo and M. Peng, "Efficient mobility management in mobile edge computing networks: Joint handover and service migration," *IEEE Internet Things J.*, 2023.

[13] Y. Sun, S. Zhou, and J. Xu, "Emm: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2637–2646, 2017.

[14] L. Jiao, A. M. Tulino, J. Llorca, Y. Jin, and A. Sala, "Smoothed online resource allocation in multi-tier distributed cloud networks," *IEEE/ACM Trans. Netw.*, vol. 25, no. 4, pp. 2556–2570, 2017.

[15] Y. Han, S. Shen, X. Wang, S. Wang, and V. C. Leung, "Tailored learning-based scheduling for kubernetes-oriented edge-cloud system," in *IEEE INFOCOM*, 2021, pp. 1–10.

[16] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," vol. 518, no. 7540, pp. 529–533, 2015.

[17] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in neural information processing systems (NeurIPS)*, 2017.

[18] M. K. Müller, F. Ademaj, T. Dittrich, A. Fastenbauer, B. Ramos Elbal, A. Nabavi, L. Nagel, S. Schwarz, and M. Rupp, "Flexible multi-node simulation of cellular mobile communications: the vienna 5g system level simulator," *EURASIP J. Wireless Commun. Netw.*, vol. 2018, no. 1, pp. 1–17, 2018.

[19] Alibaba cluster trace. [Online]. Available: https://github.com/alibaba/clusterdata

[20] Google kubernetes. [Online]. Available: https://kubernetes.io

[21] S. Chen, L. Wang, and F. Liu, "Optimal admission control mechanism design for time-sensitive services in edge computing," in *IEEE INFOCOM*, 2022.

[22] X. Huang, B. Zhang, and C. Li, "Platform profit maximization on service provisioning in mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 13 364–13 376, 2021.

[23] S. Zang, W. Bao, P. L. Yeoh, B. Vucetic, and Y. Li, "Soar: Smart online aggregated reservation for mobile edge computing brokerage services," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 527–540, 2021.

[24] B. Baek, J. Lee, Y. Peng, and S. Park, "Three dynamic pricing schemes for resource allocation of edge computing for iot environment," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4292–4303, 2020.

[25] Pricing calculator of microsoft azure. [Online]. Available: https://azure.microsoft.com/en-gb/pricing/calculator/

[26] L. Wang, L. Jiao, J. Li, J. Gedeon, and M. Mühlhäuser, "Moera: Mobility-agnostic online resource allocation for edge computing," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1843–1856, 2018.

[27] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. Tenenbaum, "Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation," in *Advances in neural information processing systems (NeurIPS)*, 2016.

[28] F. A. Oliehoek, *Decentralized pomdps*. Springer, 2012.

[29] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," in *Advances in neural information processing systems (NeurIPS)*, 2022.

[30] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in neural information processing systems (NeurIPS)*, 1999.

[31] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[32] Kubernetes placement rule-horizontal pod autoscaling. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale

[33] H. Ma, Z. Zhou, and X. Chen, "Leveraging the power of prediction: Predictive service placement for latency-sensitive mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6454–6468, 2020.

[34] F. Kelly, "Charging and rate control for elastic traffic," *European transactions on Telecommunications*, vol. 8, no. 1, pp. 33–37, 1997.

[35] H. Aziz, X. Huang, N. Mattei, and E. Segal-Halevi, "The constrained round robin algorithm for fair and efficient allocation," *arXiv preprint arXiv:1908.00161*, 2019.

[36] T. Ouyang, R. Li, X. Chen, Z. Zhou, and X. Tang, "Adaptive user-managed service placement for mobile edge computing: An online learning approach," in *IEEE INFOCOM*, 2019.

[37] D. Bhatta and L. Mashayekhy, "A bifactor approximation algorithm for cloudlet placement in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1787–1798, 2021.

[38] Z. Zhou, Q. Wu, and X. Chen, "Online orchestration of cross-edge service function chaining for cost-efficient edge computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 8, pp. 1866–1880, 2019.

[39] T. Kim, S. D. Sathyanarayana, S. Chen, Y. Im, X. Zhang, S. Ha, and C. Joe-Wong, "Modems: Optimizing edge computing migrations for user mobility," in *IEEE INFOCOM*, 2022.

[40] S.-H. Kim, S. Park, M. Chen, and C.-H. Youn, "An optimal pricing scheme for the energy-efficient mobile edge computation offloading with ofdma," *IEEE Commun. Lett.*, vol. 22, no. 9, pp. 1922–1925, 2018.

[41] M. Liu and Y. Liu, "Price-based distributed offloading for mobile-edge computing with computation capacity constraints," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 420–423, 2017.

[42] J. Yan, S. Bi, L. Duan, and Y.-J. A. Zhang, "Pricing-driven service caching and task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4495–4512, 2021.

[43] T. Bahreini, H. Badri, and D. Grosu, "Mechanisms for resource allocation and pricing in mobile edge computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 667–682, 2022.

[44] L. Ma, X. Wang, X. Wang, L. Wang, Y. Shi, and M. Huang, "Tcda: Truthful combinatorial double auctions for mobile edge computing in industrial internet of things," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 4125–4138, 2022.

[45] Z. Liu, Y. Zhao, J. Song, C. Qiu, X. Chen, and X. Wang, "Learn to coordinate for computation offloading and resource allocation in edge computing: A rational-based distributed approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 5, pp. 3136–3151, 2021.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:03:50 UTC from IEEE Xplore. Restrictions apply.